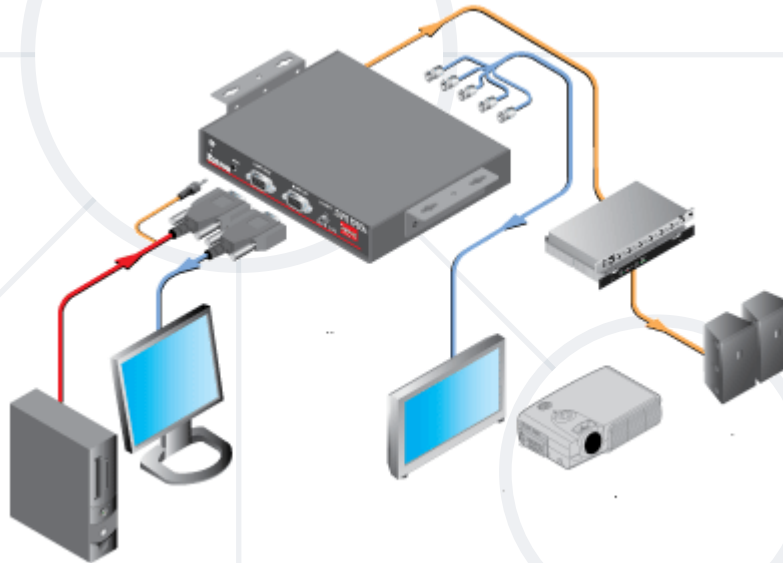


Interfaces and Abstraction

Interfaces vs Abstract Classes
Abstraction vs Encapsulation



SoftUni Team
Technical Trainers



**Software
University**



**SoftUni
Foundation**



Software University

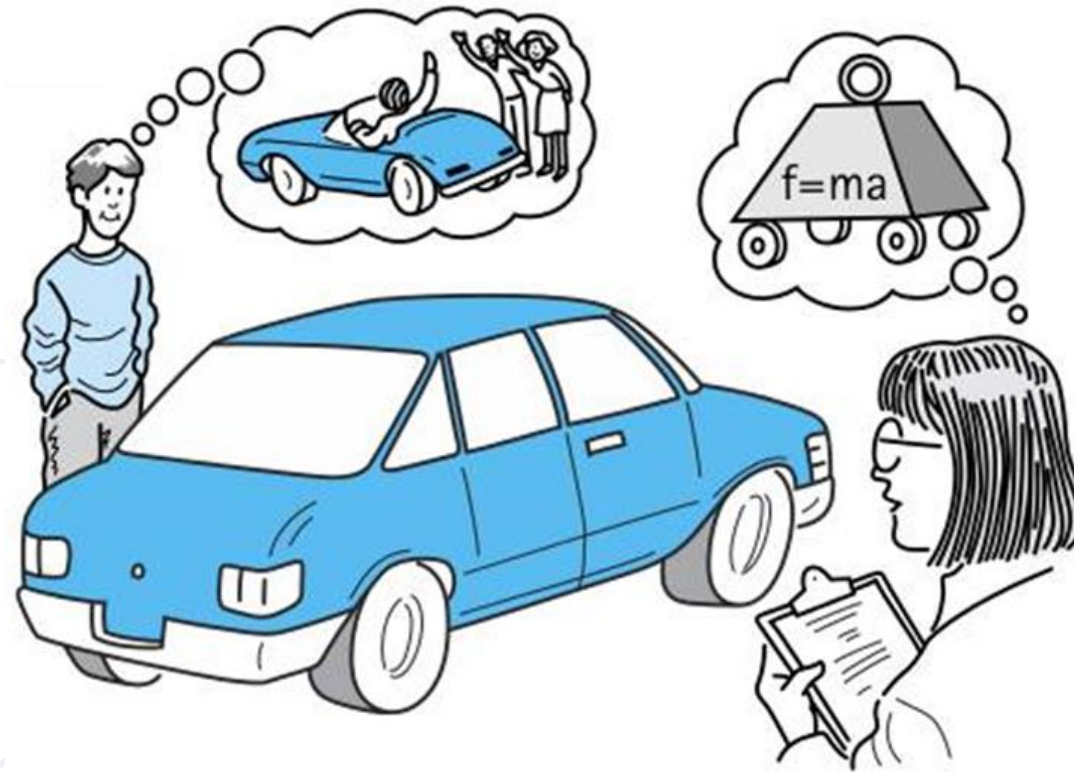
<http://softuni.bg>

1. Abstraction
2. Abstraction vs Encapsulation
3. Interfaces
4. Interfaces vs Abstract Classes



sli.do

#fund-csharp

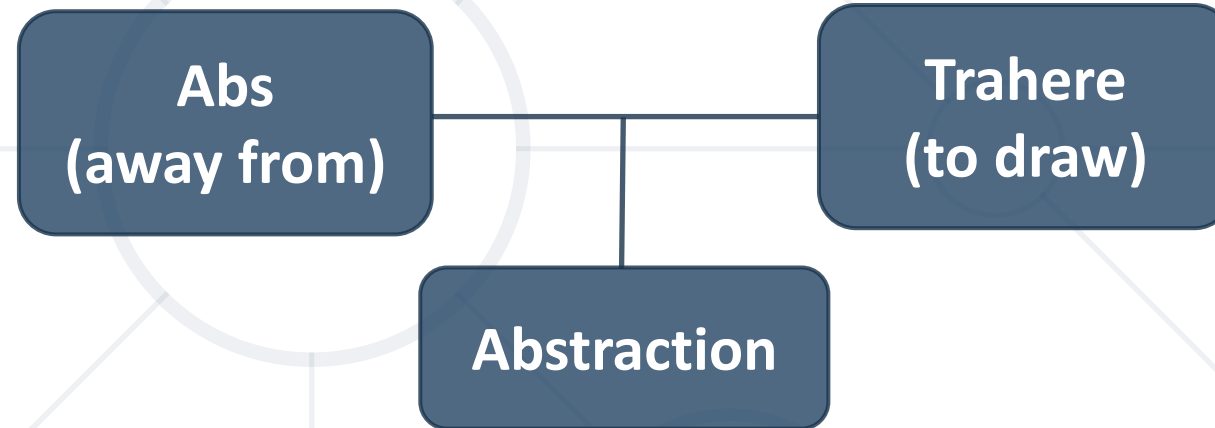


Abstraction

Working with Abstraction

What is Abstraction?

- From the Latin



- **Preserving information** that is **relevant** in a given context, and **forgetting information** that is **irrelevant** in that context.



- **Abstraction** means ignoring **irrelevant** features, properties, or functions and emphasizing the **relevant ones ...**



"Relevant" to what?

- **... relevant** to the **context** of the **project** we develop
- Abstraction helps **managing** complexity
- Abstraction lets you focus on **what the** object does instead of **how it does it.**

How do we achieve abstraction?

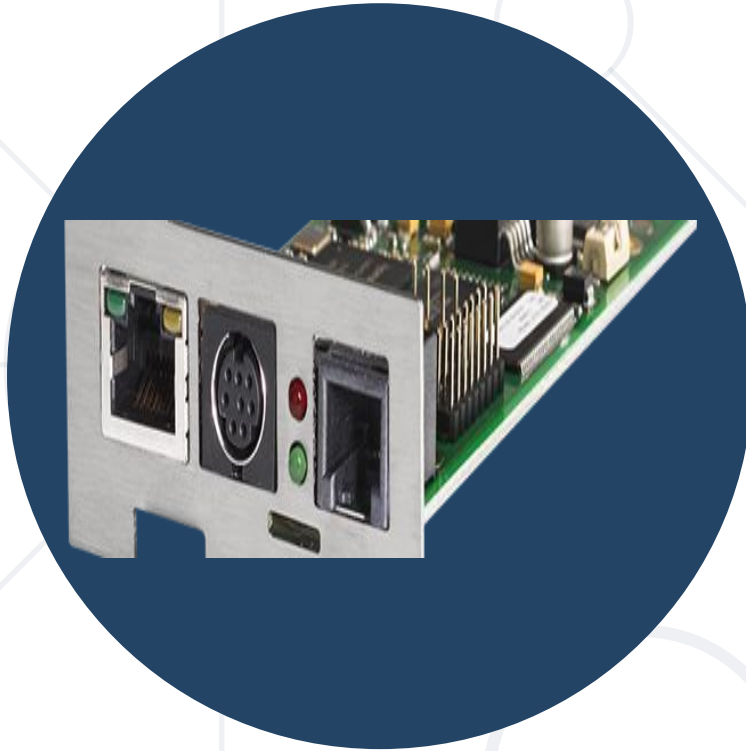
- There are two ways to achieve abstraction
 - Interfaces
 - Abstract class

```
public interface IAnimal {}  
public abstract class Mammal {}  
public class Person : Mammal, IAnimal {}
```

Abstraction vs. Encapsulation

- Abstraction
 - Process of **hiding the implementation details** and showing only functionality to the user
 - Achieved with **interfaces** and **abstract classes**
- Encapsulation
 - Used to **hide the code and data** inside a **single unit to protect the data from the outside world**
 - Achieved with **access modifiers** (private, protected, public ...)





Interfaces

Working with Interfaces

- An interface contains only the signatures of **methods**, **properties**, **events** or **indexers**
- An interface can inherit from one or more base interfaces
- When a base type list contains a base class and interfaces, the base class must come first in the list
- A class that implements an interface can explicitly implement members of that interface
 - An explicitly implemented member cannot be accessed through a class instance, but only through an instance of the interface

Interface

- Internal addition by compiler



```
public interface IPrintable {  
    void Print();  
}
```

Keyword Name

compiler

```
public interface IPrintable {  
    public abstract void Print();  
}
```

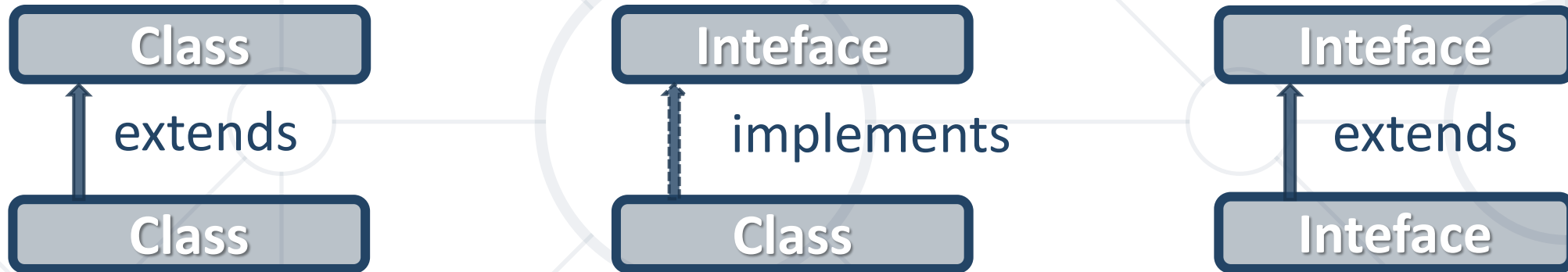
- Implementation of **Print()** is provided in class **Document**

```
public interface IPrintable {  
    void Print();  
}
```



```
class Document : IPrintable {  
    public void Print()  
    { Console.WriteLine("Hello"); }  
}
```

- Relationship between **classes** and **interfaces**

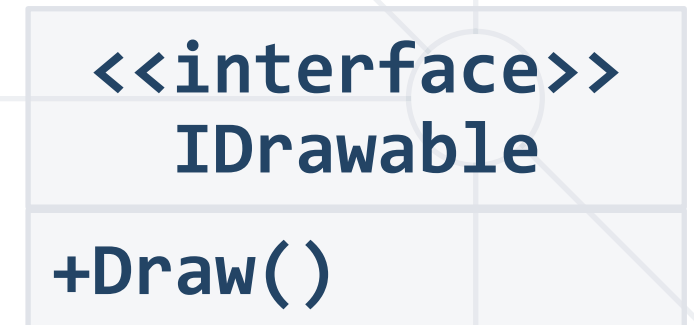
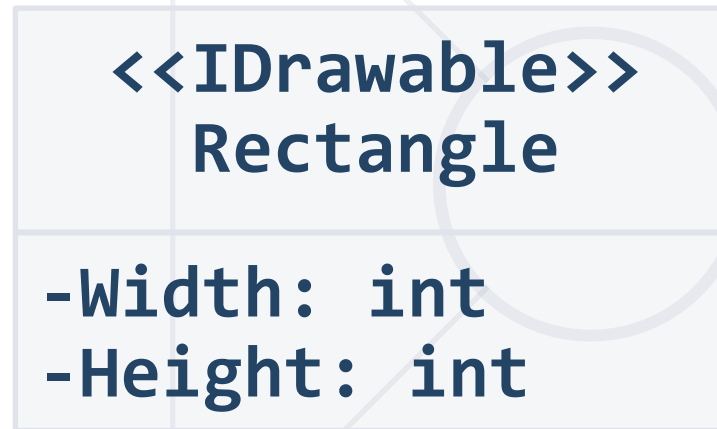
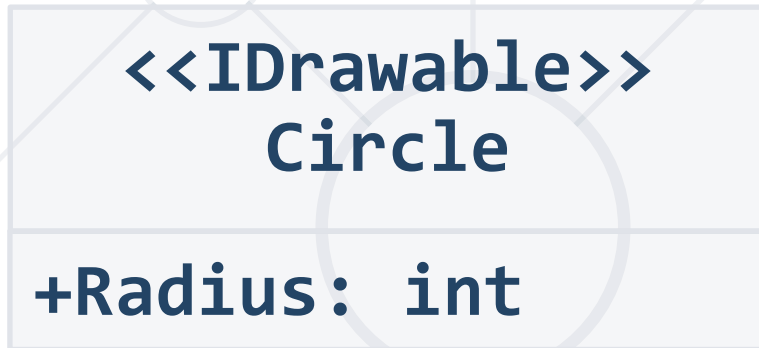


- Multiple inheritance



Problem: Shapes

- Build a project that contains an **interface** for **drawable objects**
- Implements two type of shapes: **Circle** and **Rectangle**
- Both classes have to print on the console their shape with "*"



Check your solution here: <https://judge.softuni.bg/Contests/705/Interfaces-and-Abstraction-Lab>

Solution: Shapes

```
public interface IDrawable {  
    void Draw();  
}
```

```
public class Rectangle : IDrawable {  
    //TODO: Add fields and a constructor  
    public void Draw() { //TODO: implement } }
```

```
public class Circle : IDrawable {  
    //TODO: Add fields and a constructor  
    public void Draw() { //TODO: implement } }
```

Solution: Shapes - Rectangle Draw

```
public void Draw() {  
    DrawLine(this.Width, '*', '*');  
    for (int i = 1; i < this.Height - 1; ++i)  
        DrawLine(this.Width, '*', ' ');  
    DrawLine(this.Width, '*', '*');  
}  
  
private void DrawLine(int width, char end, char mid) {  
    Console.Write(end);  
    for (int i = 1; i < width - 1; ++i)  
        Console.Write(mid);  
    Console.WriteLine(end);  
}
```

Check your solution here: <https://judge.softuni.bg/Contests/705/Interfaces-and-Abstraction-Lab>

Solution: Shapes - Circle Draw

```
double rIn = this.Radius - 0.4;
double rOut = this.Radius + 0.4;
for (double y = this.Radius; y >= -this.Radius; --y) {
    for (double x = -this.Radius; x < rOut; x += 0.5) {
        double value = x * x + y * y;
        if (value >= rIn * rIn && value <= rOut * rOut)
            Console.Write("*");
        else
            Console.Write(" ");
    }
    Console.WriteLine();
}
```

Check your solution here: <https://judge.softuni.bg/Contests/705/Interfaces-and-Abstraction-Lab>

- An abstract class cannot be **instantiated**
- An abstract class may contain **abstract** methods and accessors
- It is not possible to modify an abstract class with the **sealed** modifier
- Abstract method declarations are only permitted in abstract classes
- An abstract class must provide implementation for all interface members
 - An abstract class that implements an interface might map the interface methods onto abstract methods

Interface vs Abstract Class

- Interface
 - A class may **implement several interfaces**
 - **Cannot have access modifiers**, everything is assumed as public
 - **Cannot provide any code**, just the signature
- Abstract Class (AC)
 - May **inherit only one abstract** class
 - Can **provide implementation** and/or just the **signature** that have to be overridden
 - **Can contain access modifiers** for the fields, functions, properties



Interface vs Abstract Class (2)

- Interface

- Fields and constants **can't be defined**
- If we add **a new method we have to track down all the implementations** of the interface and **define implementation** for the new method

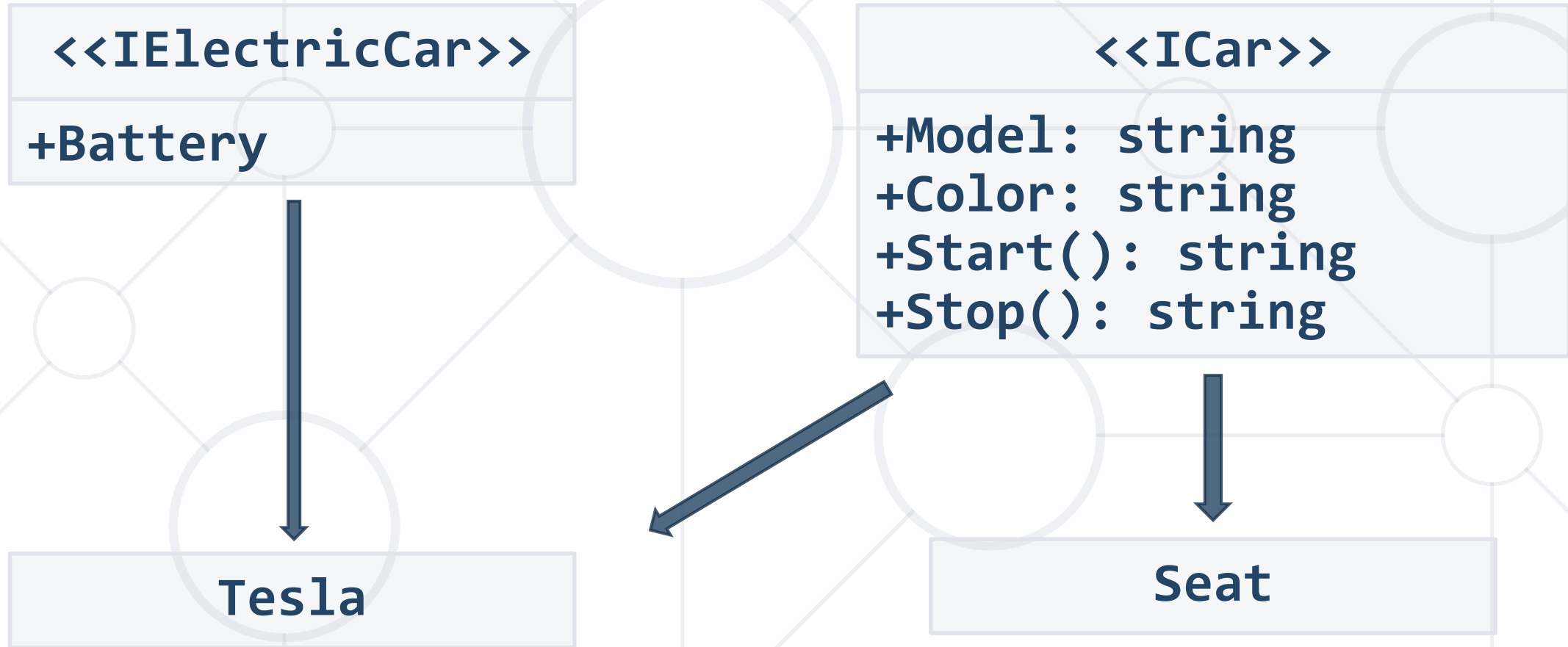
- Abstract Class

- Fields and constants **can be defined**
- If we add a **new method we** have the option of **providing default implementation** and therefore all the existing code might work properly



Problem: Cars

- Build a hierarchy of interfaces and classes



Check your solution here: <https://judge.softuni.bg/Contests/705/Interfaces-and-Abstraction-Lab>

```
public interface ICar
{
    string Model { get; }
    string Color { get; }
    string Start();
    string Stop();
}

public interface IElectricCar
{
    int Batteries { get; }
}
```

Solution: Cars (2)

```
public class Tesla : ICar, IElectricCar {  
    public string Model { get; private set; }  
    public string Color { get; private set; }  
    public int Batteries { get; private set; }  
    public Tesla(string model, string color, int batteries)  
    { //TODO: Add Logic here }  
    public string Start()  
    { //TODO: Add Logic here }  
    public string Stop()  
    { //TODO: Add Logic here }  
}
```

Check your solution here: <https://judge.softuni.bg/Contests/705/Interfaces-and-Abstraction-Lab>

Solution: Cars (3)

```
public class Seat : ICar {  
    public string Model { get; private set; }  
    public string Color { get; private set; }  
    public Tesla(string model, string color)  
    { //TODO: Add Logic here }  
    public string Start()  
    { //TODO: Add Logic here }  
    public string Stop()  
    { //TODO: Add Logic here }  
}
```

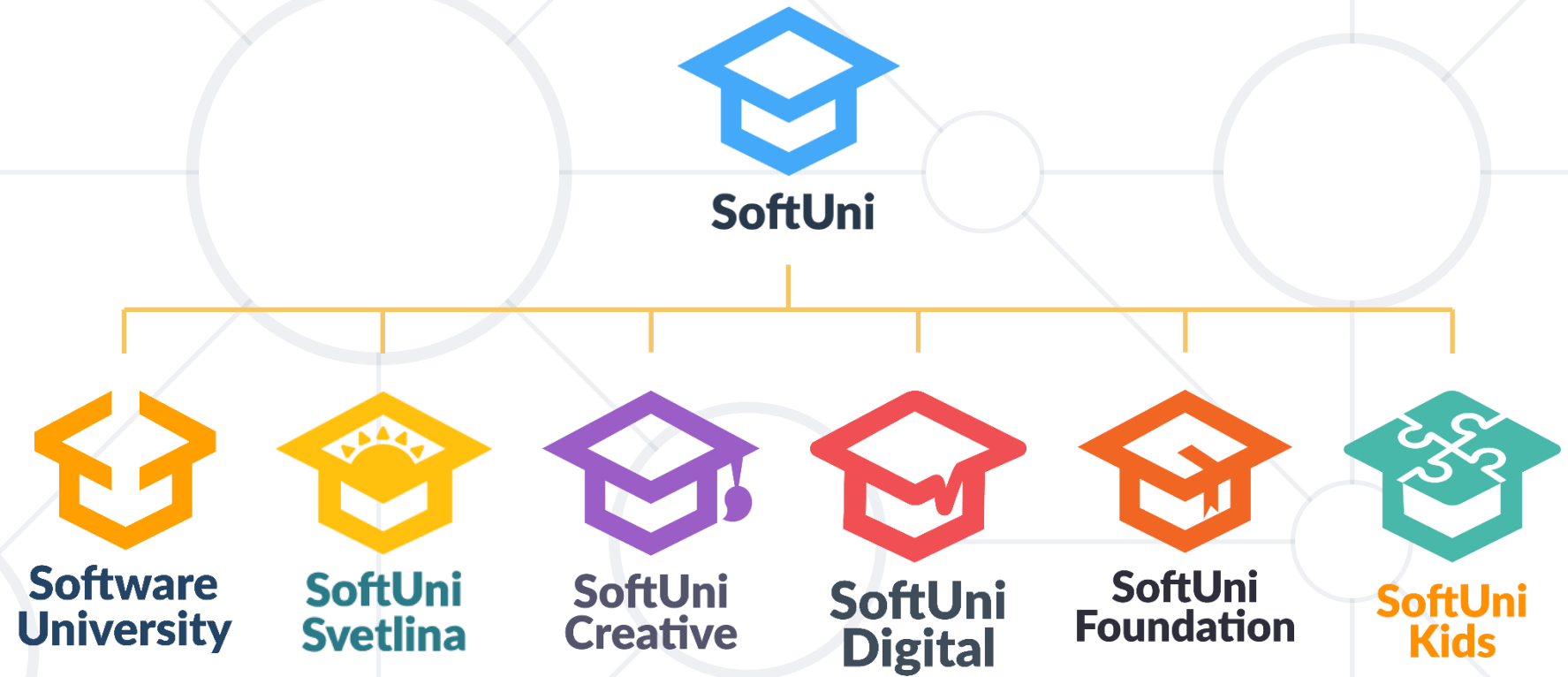



Live Exercises

- Abstraction
- How do we achieve abstraction
- Interfaces
- Abstract classes



Questions?



SoftUni Diamond Partners



XSsoftware



SBTech



telenor



SoftwareGroup
doing it right

NETPEAK



SmartIT



Postbank

Решения за твоето утре

**SUPER
HOSTING
.BG**

INDEAVR

Serving the high achievers



INFRAGISTICS®

LIEBHERR



æternity



codexio

SoftUni Organizational Partners



Trainings @ Software University (SoftUni)

- Software University – High-Quality Education and Employment Opportunities
 - softuni.bg
- Software University Foundation
 - <http://softuni.foundation/>
- Software University @ Facebook
 - facebook.com/SoftwareUniversity
- Software University Forums
 - forum.softuni.bg



- This course (slides, examples, demos, videos, homework, etc.) is licensed under the "Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International" license

